MÓDULO PROFESIONAL

Desarrollo de Aplicaciones Móviles

Análisis del entorno y desarrollo de aplicaciones móviles

Investigación, análisis y configuración de un entorno de desarrollo.

Nombre	Adrián Velasco Mañas
Curso	2º DAMP (Desarrollo de Aplicaciones Multiplataforma)
Módulo	Programación multimedia y dispositivos móviles
Entrega	04/10/2026

Índice

Introducción . . . 2

1 Análisis de las limitaciones de los dispositivos móviles . . . 2

1.1 Identificación de las principales limitaciones . . . 2
1.2 Comparativa de 5 dispositivos móviles . . . 4

2 Identificación de tecnologías de desarrollo móvil . . . 8

2.1 Java (desarrollo nativo de Android) . . . 8
2.2 Kotlin (desarrollo nativo de Android, moderno) . . . 9
2.3 Flutter . . . 10
2.4 React Native . . . 10
2.5 Swift (desarrollo nativo de iOS) . . . 11
2.6 Reflexión sobre la comparativa . . . 12

3 Configuración del entorno de desarrollo Android . . . 13

3.1 Requisitos e instalación de Android Studio . . . 13
3.2 Creación de un proyecto de prueba . . . 16
3.3 Comprobación del funcionamiento . . . 17

4 Identificación de configuraciones de dispositivos móviles . . . 20

4.1 Resolución de pantalla y densidad . . . 20
4.2 Sistema operativo y versión . . . 21
4.3 Almacenamiento interno . . . 21
4.4 Memoria RAM y potencia de procesamiento . . . 22
4.5 Conectividad . . . 23
4.6 Conclusión del apartado 4 . . . 23

5 Establecimiento de perfiles de usuario . . . 23

5.1 Tabla de perfiles . . . 23
5.2 Criterios de accesibilidad y diseño . . . 24
5.3 Reflexión final . . . 25

Conclusiones finales . . . 25

Referencias bibliográficas . . . 26

Introducción

Un móvil es un ordenador con los recursos muy justos: memoria, batería y conexión limitadas, y una pantalla pequeña que además cambia de un modelo a otro. Esta actividad investiga ese entorno y lo que implica para quien desarrolla. En el Apartado 1 identifico las limitaciones y comparo cinco teléfonos reales; en el 2 reviso las tecnologías de desarrollo (nativas y multiplataforma); en el 3 instalo Android Studio y creo un proyecto de prueba en Kotlin; en el 4 analizo las configuraciones que permiten clasificar un dispositivo, y en el 5 defino perfiles de usuario para ver que el dispositivo y la persona hay que pensarlos a la vez. Cada apartado se justifica con documentación oficial, recogida al final en las referencias.

1 Análisis de las limitaciones de los dispositivos móviles

1.1 Identificación de las principales limitaciones

Un dispositivo móvil es un ordenador con unas restricciones que no aparecen en un portátil de sobremesa: los recursos no son infinitos, la energía se agota y el usuario se lo lleva a todas partes (y eso incluye ascensores, trenes y zonas sin cobertura). Las limitaciones que explico a continuación son las que más veces se repiten cuando hay un problema real.

1 - Memoria RAM limitada

¿En qué consiste? Sabemos que la memoria RAM se utiliza para guardar datos temporales y procesos que se usan en el móvil, además de usarse para que el sistema operativo y las aplicaciones puedan funcionar correctamente, así que al desarrollar una aplicación móvil hay que saber gestionar la memoria para que el equipo pueda funcionar bien.

¿Por qué afecta a una aplicación? Una aplicación que use demasiada memoria puede hacer que el sistema se vea obligado a cerrar otras aplicaciones que estaban en segundo plano (o forzar el cierre de la aplicación).

Ejemplo práctico. Imaginemos una aplicación de edición de imágenes que carga archivos de alta resolución en la memoria. Si se utiliza en un dispositivo con poca RAM, la aplicación puede relentizar el sistema o incluso puede cerrarse porque el sistema se vea obligado a hacerlo.

¿Cómo se soluciona? La solución más evidente es optimizar el consumo de memoria pero, ¿cómo se hace? Normalmente, liberar objetos del código o recursos que no se utilizan pueden reducir el consumo, aunque hay más cosas que podemos hacer, como no cargar todas las imágenes al mismo

tiempo (tendríamos que cargarlas poco a poco), además de reducir la resolución de estas. También podemos usar una técnica llamada lazy loading, que retrasa la carga de un recurso o fragmento de código hasta el momento en que se necesita.

Lo que no es tan obvio es que el límite *no es el mismo en todos los móviles*. Android va publicando en su documentación unas tablas de «comportamiento incorrecto» (*bad behavior*) que dependen de la gama de RAM del dispositivo: para una app en primer plano el límite va desde 2,00 GB en un móvil de 4 GB hasta 4,25 GB en uno de 16 GB o más. O sea, el mismo código consume el doble de margen en un gama alta que en uno de gama baja, y el sistema lo cierra antes de que te enteres.

2 - Potencia de procesamiento limitada

¿En qué consiste? No todos los teléfonos tienen la misma capacidad de procesamiento ni todos gestionan de la misma manera los procesos asociados al rendimiento, al consumo ni a la temperatura. Un dispositivo con poca potencia de procesamiento puede tener dificultades para ejecutar operaciones dentro de nuestras aplicaciones, dificultades que en un dispositivo de alta gama no tendría.

¿Por qué afecta a una aplicación? Si dentro de nuestra aplicación se realizan demasiados cálculos, el dispositivo puede gastar muchos recursos del procesador para poder realizarlos, lo que provocaría lentitud en el móvil, poca fluidez (porque el procesador no podría ocuparse de todo al mismo tiempo y se emplearían menos recursos de lo normal en la interfaz gráfica), mucho tiempo de espera en operaciones (como abrir otras aplicaciones) y un mayor consumo de la batería (estresamos el procesador y este necesita alimentarse).

Android pone dos APIs oficiales para preguntar al sistema con qué elementos o componentes estamos jugando: `ActivityManager.getMemoryClass()` devuelve en megabytes la memoria disponible para tu aplicación y `ActivityManager.isLowRamDevice()` te dice directamente si estás en un dispositivo de gama baja. Con eso ya no hay que adivinar nada, que no está nada mal para conocer en que dispositivos usan tu aplicación.

Ejemplo práctico. Imaginemos que desarrollamos un juego 3D (con los renderizados y shaders de C++ y sus librerías) e intentamos renderizar la parte gráfica de forma constante: si tenemos un dispositivo potente no habrá problema, pero con un móvil de gama baja el juego podría presentar una bajada de FPS y podría reducirse la fluidez de forma considerable.

¿Cómo lo solucionamos? Entre otras muchas cosas (como monitorear el rendimiento en partes del mapa con alto renderizado) podemos optimizar los algoritmos de renderizado o funcionalidades con mucho cálculo (de hecho, una de las opciones que los desarrolladores de videojuegos móviles recomiendan es agrupar los objetos estáticos, para que el procesador o la gráfica pueda renderizarlos de una pasada, en vez de renderizar los objetos por separado). También podemos utilizar procesos en segundo plano para operaciones complejas, en vez de ejecutarlos en el hilo principal.

3 - Consumo de batería

¿En qué consiste? Casi todos los componentes de un móvil (por no decir todos) consumen batería, sobre todo la pantalla, el procesador, la gráfica, el GPS y la cámara, por lo que una aplicación que use todo eso sin preocuparse por la batería tendrá un problema de consumo importante.

¿Por qué afecta a una aplicación? Una aplicación con un consumo elevado de batería puede provocar una pérdida de la vida útil de un móvil pero, ¿por qué? Básicamente los móviles tienen

una salud de batería, que se calcula principalmente por los ciclos de carga del dispositivo y de la carga que se le aplique cuando se usa. Esto se traduce en que si nuestra aplicación gasta mucha batería de forma rápida, independientemente de si el móvil está cargando o no, estaríamos cargando mucho la batería, lo que perjudica a la batería del móvil (por ejemplo, si te quedas en videollamada durante varias noches con la pantalla encendida, verás como tu salud de la batería baja considerablemente, y es algo que no se puede revertir...).

Un detalle que me no me creía la primera vez: los envíos por Bluetooth siguen siendo hoy el metodo que más batería gasta del móvil, por encima del Wi-Fi. Por eso Android separa el Bluetooth clásico (transmitir audio, enviar archivos, conectar ratones) del Bluetooth Low Energy (LE), que está pensado justo para lo contrario: sensores, balizas y wearables que están casi todo el día conectados con un presupuesto de energía mínimo. Elegir el modo equivocado te puede costar horas de autonomía (y dolores de cabeza).

Ejemplo práctico. Imaginemos (otra vez) una aplicación de navegación que para consultar la ubicación necesita tener la pantalla encendida y está constantemente usando el GPS para actualizar la ubicación al instante, descargando parte del mapa alrededor del usuario: sin optimizar estos procesos, la batería del móvil se podrá gastar durante algunas pocas horas de uso constante de la aplicación.

¿Cómo lo solucionamos? Podemos evitar procesos innecesarios (tanto en primer plano como en segundo plano, no queremos procesos innecesarios), además de reducir el tiempo que tarda el GPS en actualizar la ubicación (no necesitamos una precisión milimétrica constante). También podemos reducir el uso de sensores cuando no sean necesarios y detener la aplicación cuando no se está utilizando (si el usuario minimiza la aplicación, no sería coherente seguir pidiendo constantemente la ubicación al GPS, podemos actualizar la ubicación una vez el usuario vuelva a la app).

1.2 Comparativa de 5 dispositivos móviles

Para poder comparar algo primero hay que decidir qué se compara. He cogido cinco teléfonos que sean intencionadamente distintos entre sí: tres de gama alta Android y de fabricantes distintos (Samsung, Google y OnePlus), un iPhone de gama alta, que además es el único iOS de la lista y me sirve para ver las diferencias de plataforma, y un Motorola de gama media-baja (que es el que más se parece a lo que usa la gente que no se compra un móvil cada dos años). Los datos salen de la ficha técnica de cada fabricante (casi todos...).

> ⚠ **Sobre los datos:** De todo lo que he ido metiendo en las tablas hay dos detalles que tengo que comentar: La primera es que **Apple no publica ni los mAh de la batería ni los GB de RAM del iPhone 17 Pro Max (muy tacaños por su parte)**; esas dos cifras vienen de desmontajes de terceros. La segunda es el **RAM Boost del Moto G47**, que no es RAM real: Motorola usa parte del almacenamiento interno como memoria virtual para anunciar hasta 24 GB. ¿Sirve para algo? si, pero no es lo mismo que 24 GB de RAM reales (habia que aclararlo).

Tabla 1.2.1 — A. Capacidad de procesamiento

Cuadro 1. Comparativa de capacidad de procesamiento

	Samsung S26 Ultra	Pixel 10 Pro XL	OnePlus 13	iPhone 17 Pro Max	Moto G47
Procesador / SoC	Qualcomm Snapdragon 8 Elite Gen 5 for Galaxy (TSMC 3 nm)	Google Tensor G5 (TSMC 3 nm) + coprocesador Titan M2	Qualcomm Snapdragon 8 Elite (TSMC 3 nm), módem X80	Apple A19 Pro (TSMC 3 nm, según terceros)	MediaTek Dimensity 6300 (6 nm)
Núcleos	Octa-core: 2× Oryon V3 a 4,74 GHz + 6× Oryon V3 a 3,62 GHz	Octa-core: 1× Cortex-X4 3,78 GHz + 5× Cortex-A725 3,05 GHz + 2× A520 2,25 GHz	Octa-core: 2× Oryon a 4,32 GHz + 6× Oryon a 3,53 GHz	Hexa-core: 2 rendimiento + 4 eficiencia	Octa-core: 2× A76 2,4 GHz + 6× A55 2,0 GHz
GPU	Adreno 840	PowerVR DXT-48-1536	Adreno 830	6 núcleos con ray tracing	Arm Mali-G57 M2C
RAM	12 GB (256/512 GB) · 16 GB (1 TB)	16 GB en todas las versiones	12 / 16 GB LPDDR5X	12 GB (no publicado)	8 GB LPDDR4X (+16 GB virtuales)
Almacenamiento	256 GB / 512 GB / 1 TB (UFS 4.x)	256 GB / 512 GB / 1 TB (UFS 4.0)	256 GB / 512 GB (UFS 4.0)	256 GB / 512 GB / 1 TB / 2 TB	128 GB + microSD hasta 1 TB
Ampliable	No	No	No	No	**Sí (único de los cinco)**

Tabla 1.2.2 — B. Tamaño

Cuadro 2. Comparativa de tamaño y pantalla

	Samsung	Pixel 10 Pro XL	OnePlus 13	iPhone 17 Pro Max	Moto G47
Pulgadas	6,9”	6,8”	6,82”	6,9”	6,67”
Resolución	1440×3120 (QHD+)	1344×2992 (QHD+)	1440×3168 (QHD+)	1320×2868	**720×1604 (HD+)**
Densidad aprox.	∼500 ppi	∼486 ppi	∼510 ppi	460 ppi	**264 ppi**

	Samsung	Pixel 10 Pro XL	OnePlus 13	iPhone 17 Pro Max	Moto G47
Tecnología	Dynamic AMOLED 2X (LTPO), 2.600 nits, Gorilla Armor 2	LTPO OLED, 3.300 nits, Victus 2	ProXDR LTPO 4.1, 4.500 nits, 10 bits	Super Retina XDR OLED, 3.000 nits	**LCD de 8 bits**, 1.050 nits
Refresco	120 Hz adaptativo	1-120 Hz	1-120 Hz dinámico	ProMotion hasta 120 Hz	120 Hz
Dimensiones	163,6×78,1×7 mm	162,8×76,6×8 mm	162,9×76,5×8 mm	163,4×78,0×8 mm	166,2×76,5×7,85 mm
Peso	214 g	232 g	213 g	**233 g (el más pesado)**	**191 g (el más ligero)**

Tabla 1.2.3 — C. Autonomía

Cuadro 3. Comparativa de autonomía y carga

	Samsung	Pixel 10 Pro XL	OnePlus 13	iPhone 17 Pro Max	Moto G47
Batería	5.000 mAh	5.200 mAh	**6.000 mAh (el más grande)**	~5.088 mAh (terceros)	5.200 mAh
Carga con cable	60 W (75 % en 30 min)	45 W (70 % en 30 min)	**80 W, hasta 100 W con adaptador**	50 % en 20 min con 40 W	20 W
Carga inalámbrica	25 W	25 W (con imanes)	**50 W**	MagSafe 25 W / Qi2 25 W	**No tiene**
Declaración	Hasta **31 h** de vídeo	Más de **24 h**	**No publica horas**	Hasta **39 h** de vídeo	Hasta **43 h** (varía)
Extra	Carga inversa 4,5 W	Carga inversa 10 W	Carga inversa 10 W	Sin cargador en la caja	Carga inversa 6 W

Tabla 1.2.4 — D. Conectividad

Cuadro 4. Comparativa de conectividad

	Samsung	Pixel 10 Pro XL	OnePlus 13	iPhone 17 Pro Max	Moto G47
Redes móviles	2G/3G/4G/5G (SA y NSA) + mmWave EE.UU.	2G/3G/4G/5G (SA y NSA)	2G/3G/4G/5G (solo Sub-6)	5G sub-6 + mmWave, LTE gigabit	2G/3G/4G/5G (solo Sub-6)
Wi-Fi	Wi-Fi 7 (802.11be)	Wi-Fi 7 (802.11be)	Wi-Fi 7 (802.11be)	Wi-Fi 7 (802.11be)	**Wi-Fi 5 (802.11ac)**
Bluetooth	**6.0**	**6.0**	5,4	**6**	5,4
NFC	Sí (con UWB)	Sí	Sí (con eSE)	Sí (modo lector)	Sí (según mercado)
UWB	Sí	Sí	**No**	Sí (chip 2.ª gen.)	**No**
Otros detalles	Satélite (SOS y mensajes)	Satélite SOS, Thread/Matter	**Mando IR**, BeaconLink	Satélite, Thread, USB-C 10 Gb/s	**Jack 3,5 mm**, IP64, MIL-STD-810H
SIM	Nano-SIM + eSIM	Nano-SIM + eSIM	Doble nano-SIM	**Solo eSIM EE.UU.**, nano+eSIM Europa	2× nano-SIM, **sin eSIM**
USB	USB-C 3.2	USB-C 3.2	USB 3.2	USB-C hasta 10 Gb/s	USB-C 2.0

Conclusión de la comparativa

¿Qué dispositivo presenta una mayor capacidad de procesamiento?

Depende (como todo): si hablamos de frecuencia del procesador y su arquitectura, ganaria el Samsung galaxy-s26-ultra; pero si hablamos de cantidad de RAM (real) ganaria el Pixel. Lo unico que tenemos seguro de esta comparación es que Motorola se queda al final de la lista por obvias razones.

¿Cuál tiene una pantalla de mayor tamaño?

El Samsung Galaxy S26 Ultra y el iPhone 17 Pro Max empatan en 6,9 pulgadas, y en centímetros son prácticamente el mismo teléfono (163,6 mm frente a 163,4 mm de alto). Pisandole los talones tenemos el OnePlus 13 con 6,82”, el Pixel 10 Pro XL con 6,8” y el Motorola Moto G47 con 6,67”.

¿Qué diferencias encuentras en la capacidad de las baterías?

Principalmente veo que en capacidad gana el OnePlus 13 con sus 6.000 mAh, seguido del Pixel y el Motorola (los dos con 5.200 mAh); el Iphone con sus 5.088 y el Samsung, en última posicion con 5.000 mAh clavados. Tambien podemos destacar que la velocidad de carga de los dispositivos se llevan mucha diferencia, como por ejemplo el Motorola con 20 W y el OnePlus con 80 W (y 100 W si compras el cargador aparte y te sobra el dinero).

¿Qué dispositivos ofrecen más opciones de conectividad?

El más completo es el Samsung Galaxy S26 Ultra: 5G con mmWave, Wi-Fi 7, Bluetooth 6.0, NFC, UWB, USB 3.2, eSIM y satélite. El iPhone 17 Pro Max y el Pixel 10 Pro XL están casi a su nivel. El OnePlus 13 es el raro de la lista, porque trae mando IR y se queda sin UWB. Y el Motorola Moto G47 es el que más se aleja de todo: Wi-Fi 5 en vez de Wi-Fi 7, sin UWB, sin eSIM, sin satélite y USB 2.0.

¿Cómo podrían influir estas diferencias en la ejecución de una misma aplicación móvil?

Ejecutando la misma aplicación en los 5 dispositivos, lo que cambia son los componentes que unos tienen y otros no (como el NFC). Si la app usa UWB, funcionara en 3 de los 5 dispositivos; si la aplicacion usa 5G para precargar un video, en el Motorola habira que desarrollarlo de otra forma.

2 Identificación de tecnologías de desarrollo móvil

Elegir tecnología me parece la decisión más importante de un proyecto y también la más cara de revertir: cambiar de framework con la app a medio hacer es casi empezar otra vez. Por eso en este apartado quiero explicar por qué existe cada una y en qué situación concreta sería la mejor elección.

He mirado cinco: las nativas de Android (Java y Kotlin), las multiplataforma (Flutter y React Native) y la nativa de iOS (Swift).

2.1 Java (desarrollo nativo de Android)

Nombre de la tecnología. Java (El lenguaje original para el desarrollo en Android).

Lenguaje. Java.

Tipo de desarrollo. Nativo. El código compila a bytecode y se ejecuta en la máquina virtual de Android sobre las APIs del sistema.

Sistemas operativos compatibles. Android solamente (no funciona para iOS).

Principales características.

- Acceso completo y directo a todas las APIs del sistema (cámara, GPS, Bluetooth, sensores, notificaciones, Wear OS...).
- Interfaz de dos formas: con **vistas XML** (`res/layout/*.xml`) o de forma declarativa con **Jetpack Compose**.
- Herramienta oficial: **Android Studio**, que trae compilador (Gradle), emulador, inspector de memoria y de red, y un perfilador de CPU.

- Un dato que me llamó la atención: en mayo de 2026 Google publicó una entrada de blog titulada literalmente «Android UI development is Compose-first». El sistema de vistas clásico entra en modo mantenimiento.

Ventajas. Rendimiento máximo, porque no hay capas intermedias; cobertura total de la plataforma; ecosistema maduro y estabilidad a largo plazo.

Desventajas. Solo sirve para Android y su mantenimiento no es tan comodo como otros lenguajes.

Ejemplo o situación adecuada. Una app de banco (como es el caso de CaixaBank) o de salud que necesite leer el NFC de la tarjeta sanitaria, abrir la cámara para escanear recetas y funcionar en el *longtail* de Android barato, donde el rendimiento y el tamaño del APK importan mucho.

2.2 Kotlin (desarrollo nativo de Android, moderno)

Nombre de la tecnología. Kotlin (Mi favorito de los lenguajes de desarrollo en Android).

Lenguaje. Kotlin.

Tipo de desarrollo. Nativo para Android y además multiplataforma mediante **Kotlin Multiplatform (KMP)**.

Sistemas operativos compatibles. Android de forma nativa; y mediante KMP, la misma lógica de negocio se puede compartir con iOS, macOS, Windows, Linux, web (Wasm/JS), tvOS y watchOS (y compatible con Java).

Principales características.

- Conciso y seguro: tiene tipado nulo, que eso ya elimina muchísimos errores en tiempo de ejecución.
- Coroutines, que es la forma oficial de hacer asincronía sin bloquear la interfaz.
- Interoperabilidad total con Java: puedes llamar a Java desde Kotlin y al revés.
- Jetpack Compose para la interfaz moderna declarativa (Aunque también está Compose Multi-platform).
- Google mantiene una estrategia **Kotlin-first**: la documentación nueva sale primero en Kotlin.

Ventajas. Menos código y menos errores que Java; las coroutines resuelven muy bien el trabajo en segundo plano y KMP permite reutilizar la lógica entre Android e iOS.

Desventajas. Curva de aprendizaje mas alta que en Java (por lo menos en mi experiencia); KMP todavía está madurando y dependende mucho de Google.

Ejemplo o situación adecuada. Una app de comercio electrónico que nace para Android, tiene una lógica de precios, stock y carrito complicada, y en un año quiere publicar en iOS. Con KMP se comparte el 70-80 % del código de negocio.

2.3 Flutter

Nombre de la tecnología. Flutter (El que sirve en cualquier sitio).

Lenguaje. Dart.

Tipo de desarrollo. Multiplataforma, con una única base de código.

Sistemas operativos compatibles. Android (API 24+), iOS/iPadOS (15.0+), Windows, macOS, Linux y web. Es la lista más amplia de las cinco.

Principales características.

- Flutter no usa los botones ni las vistas nativas: dibuja todos los píxeles él mismo a través de su propio árbol de widgets.
- Impeller es su renderizador por defecto desde la 3.27 en iOS, y en Android a partir de la API 29.
- Hot reload de verdad: cambias un `Text` y lo ves en el móvil en menos de un segundo.
- Ecosistema en `pub.dev` con más de 1.300 millones de descargas en los últimos 30 días.
- Versión actual: Flutter 3.47 (agosto de 2026).

Ventajas. Una sola base de código para todo, es decir, interfaz visual idéntica en todas partes; rendimiento muy cercano al nativo y hot reload al instante.

Desventajas. La aplicación resultante pesa más, dependencdes de plugins; el aspecto nativo no es exacto; Dart es un lenguaje relativamente nuevo; riesgo de quedarte atado a una sola plataforma.

Ejemplo o situación adecuada. Una startup que quiere lanzar la misma app en iOS, Android, web y escritorio en tres meses con un equipo pequeño.

2.4 React Native

Nombre de la tecnología. React Native, el framework multiplataforma de Meta.

Lenguaje. JavaScript o TypeScript. (Si me preguntan a mi, mejor Typescript)

Tipo de desarrollo. Multiplataforma, sobre todo Android e iOS.

Sistemas operativos compatibles. Android, iOS y, mediante proyectos externos (*out-of-tree*), macOS, Windows, visionOS, tvOS, web y HarmonyOS.

Principales características.

- Al revés que Flutter, sí usa las vistas nativas de cada sistema.
- La Nueva Arquitectura (Fabric y Turbo Native Modules) es el modo por defecto desde la 0.76, y desde la 0.82 (octubre de 2025) es la única arquitectura.
- Fast Refresh activado por defecto.
- Versión actual: React Native 0.87 (agosto de 2026), que exige Node 22+.

- Ecosistema npm: cualquier librería de JavaScript probablemente ya tiene versiones LTS.

Ventajas. Enorme mercado de desarrolladores (gracias a su ecosistema npm gigantesco); integración incremental y buen rendimiento con la Nueva Arquitectura.

Desventajas. Los módulos nativos son fuente de problemas; las actualizaciones mayores son complejas, el rendimiento es variable; depurar es más complejo; tiene menos presencia que Flutter en el mercado de frameworks.

Ejemplo o situación adecuada. Una empresa que ya tiene un equipo web grande y quiere sacar la app móvil sin contratar gente nueva.

2.5 Swift (desarrollo nativo de iOS)

Nombre de la tecnología. Swift, el lenguaje oficial de Apple.

Lenguaje. Swift.

Tipo de desarrollo. Nativo, y solo del ecosistema Apple.

Sistemas operativos compatibles. iOS, iPadOS, macOS, tvOS, visionOS y watchOS. No hay versión para Android.

Principales características.

- Dos formas de hacer interfaz: SwiftUI (declarativo) y UIKit (imperativo). Apple dice que conviven y que se adopte «a tu propio ritmo».
- Concurrencia moderna: la versión actual es Swift 6.4 (septiembre de 2026), con *strict concurrency*.
- Xcode Previews: ves la pantalla en el canvas lateral sin arrancar la app.
- Integración profunda: Core Data, SwiftUI, Metal, Core ML, HealthKit, ARKit…
- Seguridad de memoria gestionada (ARC).

Ventajas. Máximo rendimiento y fidelidad en plataforma Apple; acceso completo a cada API; lenguaje moderno y seguro; herramientas excelentes (Xcode + SwiftUI Previews); soporte a largo plazo coherente.

Desventajas. Lock-in total; cuesta aprender dos cosas (SwiftUI y UIKit); ecosistema de herramientas más estrecho; la API evoluciona muy rápido.

Ejemplo o situación adecuada. Cualquier app de calidad o de prestigio en el ecosistema Apple. Siri, Things, Fantastical, Bear, Overcast… todas están escritas en Swift.

Tabla 2.1 — Comparativa de las cinco tecnologías

Cuadro 5. Comparativa técnica de las cinco tecnologías

	Java	Kotlin	Flutter	React Native	Swift
Lenguaje	Java	Kotlin	Dart	JS/TS	Swift
Tipo	Nativo	Nativo (+KMP)	Multiplataforma	Multiplataforma	Nativo
Android	Sí	Sí	Sí	Sí	**No**
iOS	**No**	Con KMP	Sí	Sí	Sí
Escritorio	**No**	Con KMP	Win/mac/Linux	Proyectos externos	macOS
Web	**No**	Experimental	Sí	Proyectos externos	**No**
Cómo dibuja la UI	Vistas XML o Compose	Compose o XML	Motor propio	Vistas nativas	SwiftUI o UIKit
Rendimiento	Máximo	Máximo	Muy alto	Alto	Máximo
Tamaño de la app	Bajo	Bajo	**Alto**	Medio	Bajo
Hot reload	Limitado	Sí (Compose)	Excelente	Fast Refresh	Previews Xcode
Curva de aprendizaje	Alta	Media	Media	Media (web)	Media

2.6 Reflexión sobre la comparativa

¿Qué diferencias existen entre el desarrollo nativo y el multiplataforma?

La diferencia realamente no es tanto técnica, sino de de como se organizan a nivel de componentes. En el nativo la aplicación usa directamente los componentes del sistema; en el multiplataforma hay una capa intermedia que traduce, y de esa capa salen todas las demás diferencias:

- El aspecto visual. El nativo se ajusta mas al entorno.
- El rendimiento y el consumo. El nativo es más eficiente, siempre.
- El acceso al hardware. El nativo llega a todo; el multiplataforma, a lo que alguien haya escrito un binding.
- El tamaño de la aplicación. El nativo pesa menos.
- El coste de desarrollo. Aquí el multiplataforma gana sin discusión, pero con una condición: si luego necesitas algo muy nativo en una de las dos plataformas, la ventaja se evapora.

Después de comparar las seis, mi conclusión es que la decisión no es técnica, es organizativa. La pregunta que de verdad importa es «¿qué equipo tengo y qué necesitan mis usuarios?».

¿Qué ventajas puede ofrecer una aplicación desarrollada específicamente para Android?

Principalmente, cuatro:

1. Acceso completo a todas las APIs de Android sin intermediarios.
2. Rendimiento y consumo óptimos.

3. Tamaño de descarga menor.
4. Publicación inmediata en un solo sitio.

¿Qué beneficios puede aportar una tecnología que permita desarrollar para varios sistemas operativos?

Principalmente el coste: el ahorro típico está entre el 30 % y el 50 % del coste total de desarrollo, no el 80 % que se repite mucho y que no se cumple. Luego la velocidad de salida al mercado, un único equipo de mantenimiento, consistencia del producto y de los valores, y actualizaciones simultáneas.

¿Qué factores debería tener en cuenta una empresa antes de elegir una tecnología?

Si tuviera que ordenarlos por peso: dónde están nuestros usuarios y qué necesitan; qué sabe hacer el equipo y qué quiere aprender; requisitos técnicos muy específicos; presupuesto y plazo; mantenimiento a 5 años vista; riesgo de proveedor; y por último accesibilidad y regulación.

3 Configuración del entorno de desarrollo Android

3.1 Requisitos e instalación de Android Studio

Paso 0 — Comprobar los requisitos del equipo. Antes de descargar nada hay que mirar lo que publica Google, porque hay una cosa que puede hacer que todo lo demás falle después. En mi caso el punto crítico es siempre el mismo: la **virtualización por hardware**. Si no está activada en la BIOS, el emulador no arranca y no hay arreglo por software.

Los requisitos oficiales para Windows son: 64 bits, 8 GB de RAM (**16 GB si se usa el emulador**; 32 GB recomendados), 8 GB de disco libre (16 GB con emulador), pantalla de 1280×800 y un procesador con virtualización activada (Intel VT-x o AMD-V).

Paso 1 — Descargar el instalador. Se descarga desde `https://developer.android.com/studio`. La versión estable es **Quail 4 | 2026.1.4 Patch 1**, y el instalador ocupa unos **1,5 GB** (pero nosotros instalaremos Rabbit 1, que es la mas nueva): Hay instaladores para Windows (`.exe`), macOS (`.dmg`), Linux (`.tar.gz`) y ChromeOS.

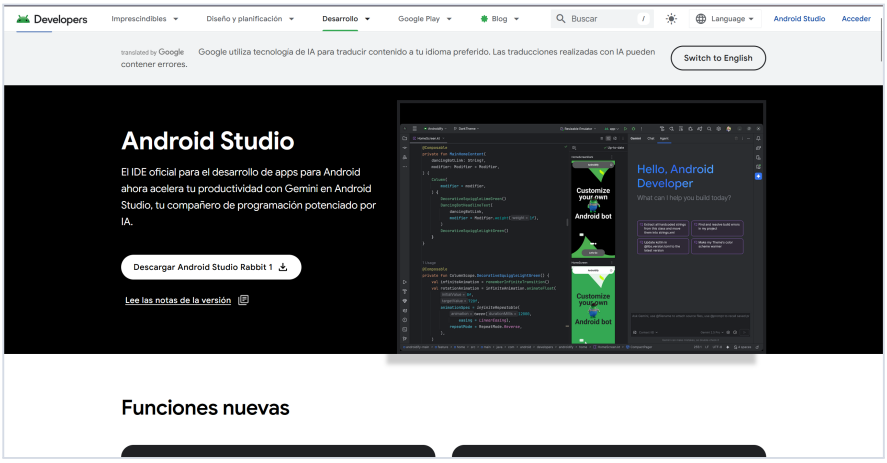


Figura 1. Instalador de Android Studio (Rabbit 1)

Paso 2 — Ejecutar el instalador. En Windows basta con hacer doble clic en el `.exe` y seguir el asistente; en macOS se arrastra el icono a Aplicaciones y en Linux se descomprime en `/opt/`.



Figura 2. Pantalla de arranque de Android Studio Rabbit 1 (2026.2.1), que confirma la versión instalada.

Paso 3 — El asistente de configuración inicial (Setup Wizard). El asistente ofrece dos modos de instalación:

- **Standard (estándard)**: instala automáticamente el conjunto recomendado.
- **Custom (personalizado)**: permite elegir componente a componente.

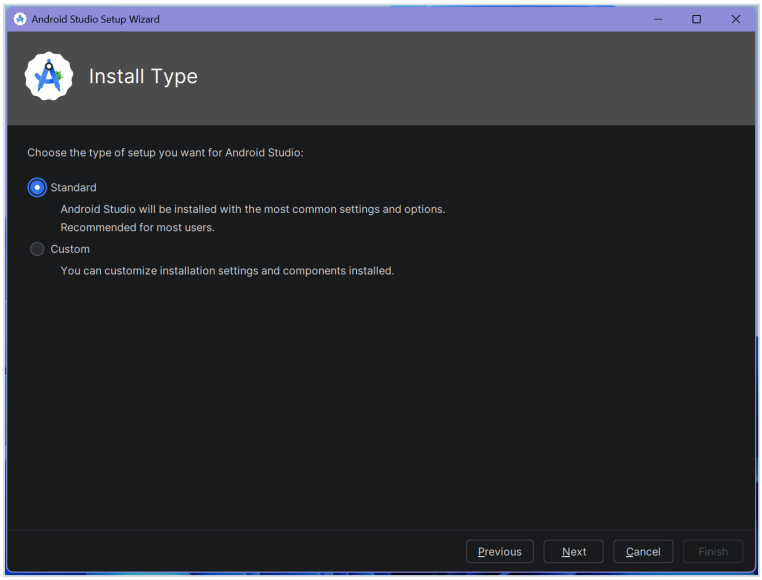


Figura 3. Asistente de configuración inicial (*Setup Wizard*): pantalla *Install Type* con el modo **Standard** seleccionado.

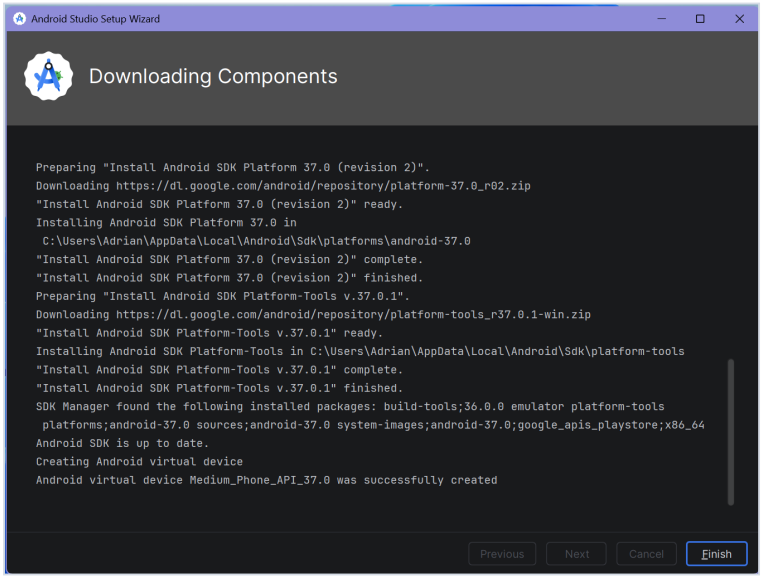


Figura 4. *Setup Wizard*: descarga de componentes terminada. El registro indica que el SDK está actualizado y que se ha creado el dispositivo virtual `Medium_Phone_API_37.0`.

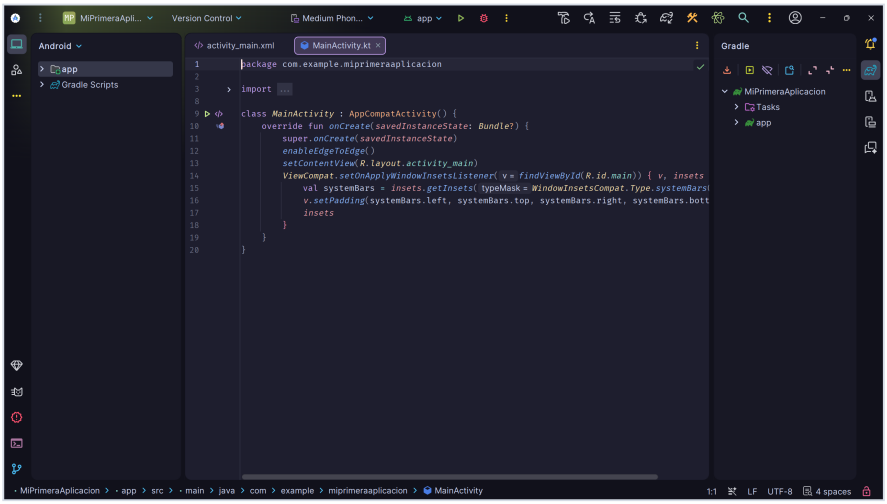


Figura 5. Pantalla principal de Android Studio con el proyecto `MiPrimeraAplicacion` abierto: panel *Project* a la izquierda, editor en el centro y panel de Gradle a la derecha.

3.2 Creación de un proyecto de prueba

Paso 1 — Abrir el asistente de nuevo proyecto. Botón **New Project** en la pantalla de bienvenida, o **File >New >New Project**.

Paso 2 — Elegir la plantilla. En el panel de plantillas, dentro de **Phone and Tablet**, hay que seleccionar **`Empty Views Activity`**.

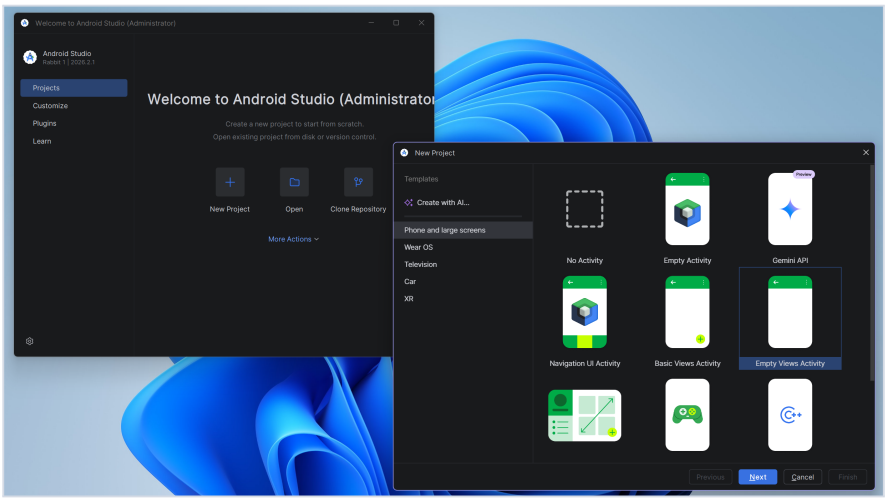


Figura 6. Asistente *New Project*: plantilla **Empty Views Activity** seleccionada en la categoría *Phone and large screens*.

Paso 3 — Configurar el proyecto.

Campo	Valor a introducir	Por qué
Name	`MiPrimeraAplicacion`	Nombre del proyecto y de la carpeta
Package name	`com.ejemplo.miprimeraaplicacic`	Identificador único. No se puede cambiar después
Save location	Carpeta con espacio	El proyecto + emulador se llevan bastante
Language	**Kotlin**	El enunciado es explícito: debe ser Kotlin, no Java
Minimum API level	**API 36 (Android 16.0 Baklava)**	Version instalada en clase

Cuadro 6. Campos de configuración del nuevo proyecto

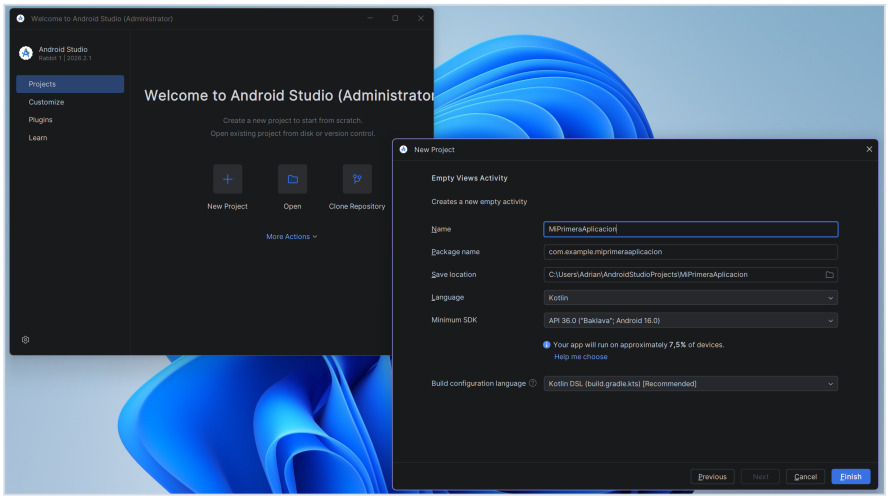


Figura 7. Asistente *New Project* con los campos rellenos: nombre `MiPrimeraAplicacion`, **Language = Kotlin** y SDK mínimo API 36.

Paso 4 — Esperar a la sincronización. Al pulsar **Finish**, Android Studio crea el proyecto y lanza una sincronización de Gradle en segundo plano. En la parte inferior se ve la barra de progreso, y conviene esperar a que termine antes de tocar nada más.



Figura 8. Panel *Build* tras crear el proyecto: la sincronización de Gradle termina con `BUILD SUCCESSFUL in 18s` y sin errores en rojo.

3.3 Comprobación del funcionamiento

Paso 1 — Localizar la estructura del proyecto. Nada más abrir el proyecto, lo primero que hago es mirar el árbol de archivos, que es donde se ve si Android Studio ha generado lo que se le ha pedido:

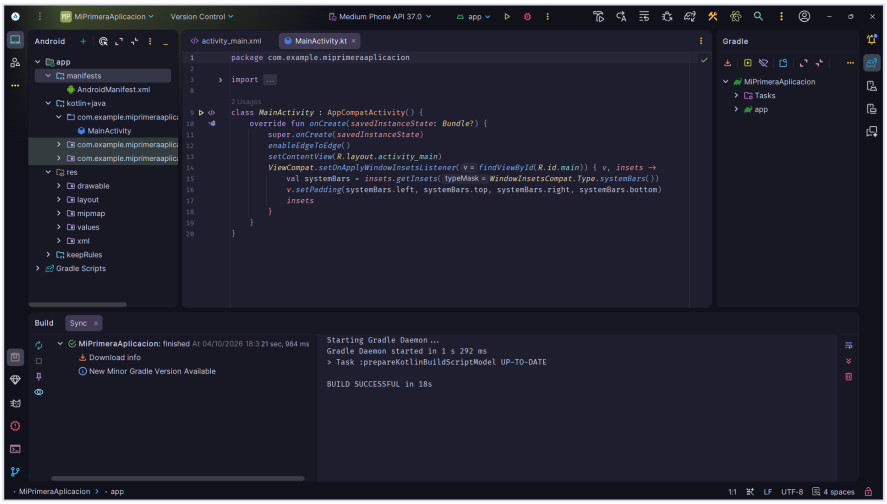


Figura 9. Android Studio con el panel *Project* desplegado (vista *Android*): `manifests`, `kotlin+java`, `res` y `Gradle Scripts`.

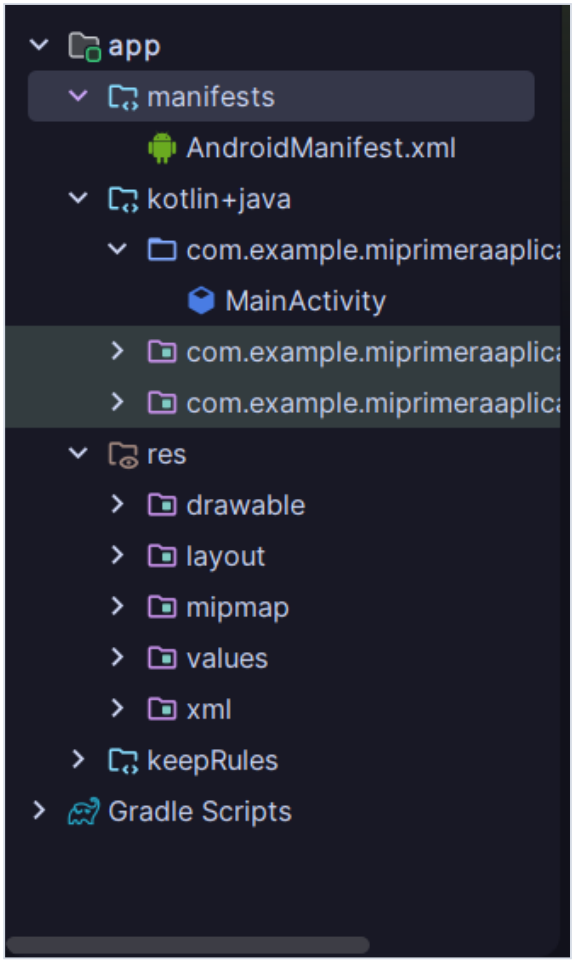


Figura 10. Detalle ampliado del árbol del proyecto: `MainActivity` dentro de `kotlin+java` y las carpetas de `res`.

Paso 2 — Identificar la actividad principal y el diseño. Son los dos archivos que demuestran que el proyecto está bien creado: `app/src/main/java/.../MainActivity.kt` (la clase `MainActivity` con su `onCreate`, que llama a `setContentView`) y `app/src/main/res/layout/activity_main.xml` (la

jerarquía de vistas).



Figura 11. Editor con `MainActivity.kt`: la extensión `.kt` y la declaración `class MainActivity : AppCompatActivity()` confirman que la actividad principal está escrita en Kotlin.

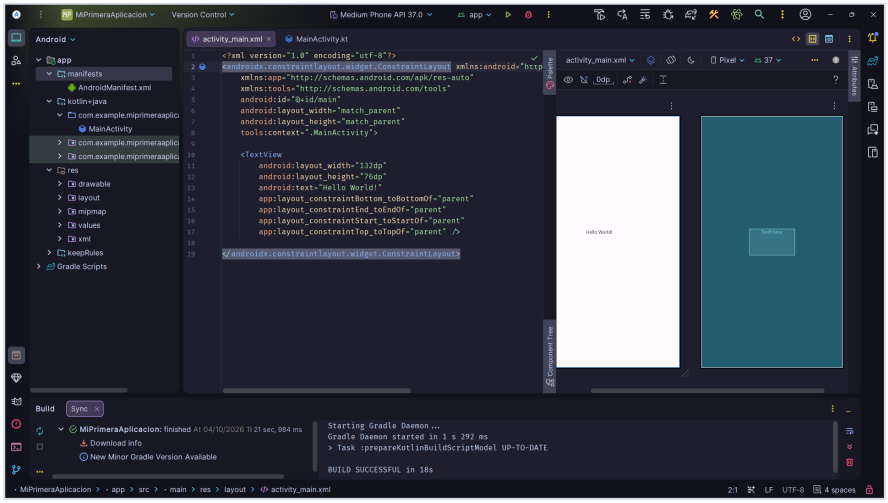


Figura 12. Archivo `activity_main.xml` en vista *Split*: el XML con la jerarquía de vistas (`ConstraintLayout` con un `TextView`) y su vista previa.

Paso 3 — Ejecutar en el emulador. Antes de nada, confirmar que la **virtualización por hardware está activada en la BIOS**:

```
emulator -accel-check
```

El dispositivo virtual se crea desde **Device Manager** → **Create Virtual Device**, eligiendo un perfil Pixel y una imagen de sistema. Para ejecutar: botón **Run** (o **Shift + F10**).

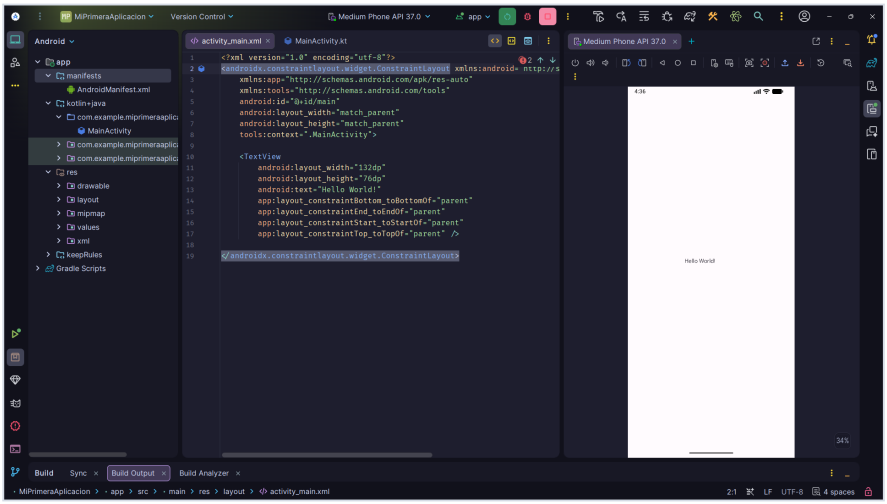


Figura 13. Aplicación ejecutada en el emulador `Medium Phone API 37.0`: se muestra el *Hello World!* definido en `activity_main.xml`.

4 Identificación de configuraciones de dispositivos móviles

Este apartado junta las limitaciones del Apartado 1 con las herramientas del Apartado 2: qué datos de un dispositivo hay que mirar para saber si una aplicación va a funcionar bien en él. Analizo cinco criterios: resolución y densidad, sistema operativo y versión, almacenamiento, RAM y procesamiento, y conectividad.

4.1 Resolución de pantalla y densidad

¿En qué consiste? Una pantalla no se mide solo en píxeles: se distinguen los **píxeles físicos**, la **densidad** (píxeles por pulgada) y los **dp** (píxeles independientes de la densidad). La relación oficial es `px = dp × (dpi / 160)`.

Valores o categorías. Android agrupa la densidad en seis cubos y el tamaño en cuatro categorías según los dp disponibles (`small` 320×426, `normal` 320×470, `large` 480×640 y `xlarge` 720×960 dp como mínimo):

¿Cómo afecta a la aplicación? El mismo número de píxeles no significa lo mismo en dos teléfonos. Para adaptar la interfaz hay que usar `dp` para los tamaños, `sp` para el texto, imágenes vectoriales y decidir el diseño según el espacio disponible y no según el modelo. Además, desde **Android 17 (API 37)** ya no se pueden bloquear la orientación ni el redimensionado en pantallas grandes (`sw` ≥ 600 dp).

Ejemplo con dos dispositivos. El Moto G47 (720×1604, 264 ppi) dibuja un icono de 48 dp con unos 79 px; el Pixel 10 Pro XL (1344×2992, 486 ppi) lo dibuja con unos 146 px. Si la app solo trae

Grupo	Nombre	dpi aprox.	Factor	Icono 48dp
`ldpi`	Baja densidad	∼120	0,75×	36×36 px
`mdpi`	Densidad media (**ref.**)	∼160	1,0×	48×48 px
`hdpi`	Alta densidad	∼240	1,5×	72×72 px
`xhdpi`	Extra alta	∼320	2,0×	96×96 px
`xxhdpi`	Extra extra alta	∼480	3,0×	144×144 px
`xxxhdpi`	Extra extra extra alta	∼640	4,0×	192×192 px

Cuadro 7. Grupos de densidad de Android

la versión pequeña del icono, en el segundo se escala y se ve borroso.

4.2 Sistema operativo y versión

¿En qué consiste? Es el sistema sobre el que corre la app: del que dependen las APIs, los permisos y el comportamiento. En Android hay dos números distintos, el **nombre de la versión** y el **nivel de API**, y no se pueden deducir uno del otro.

Valores o categorías. Los sistemas móviles principales son **Android** (muchos fabricantes, actualizaciones desiguales) e **iOS** (solo Apple, versiones muy homogéneas). En Android, las versiones y sus niveles de API son: Android 17 = 37, 16 = 36, 15 = 35, 14 = 34, 13 = 33, 12L/12 = 32/31, 11 = 30, 10 = 29, 9 = 28, 8.1/8.0 = 27/26 y 7.1/7.0 = 25/24.

¿Cómo afecta a la aplicación?

1. **Si no subes el `targetSdk`, Google Play te deja fuera.** Desde el **31 de agosto de 2026**, las apps nuevas y las actualizaciones tienen que apuntar a Android 16 (API 36) o superior.
2. **Las apps antiguas se vuelven invisibles:** para seguir apareciendo a usuarios nuevos hay que apuntar como mínimo a **Android 15 (API 35)**.
3. **El sistema cambia de comportamiento** según la versión (permisos, ejecución en segundo plano), así que la misma línea de código puede pedir cosas distintas.

En mi proyecto de prueba el SDK mínimo es el API 36 instalado en clase; en un proyecto real bajaría el `minSdk` para llegar a más móviles.

Ejemplo con dos dispositivos. Un iPhone 17 Pro Max (iOS) y un Moto G47 (Android) ni siquiera ejecutan el mismo paquete: la app se compila dos veces, y en Android además hay que decidir `minSdk` y `targetSdk`.

4.3 Almacenamiento interno

¿En qué consiste? Es el espacio donde van las apps y sus datos. Hay que distinguir la **capacidad del dispositivo** del **espacio libre** para una aplicación concreta.

Valores o categorías. Android no publica una tabla oficial de gamas de almacenamiento, así que las franjas son convención de mercado:

Gama	Total	Útil	Perfil típico
Baja	16-32 GB	8-10 GB	Móvil antiguo o de entrada
Media	64-128 GB	40-70 GB	La gama mayoritaria
Alta	256 GB - 1 TB	200 GB+	Gama alta

Cuadro 8. Gamas de almacenamiento (convención)

¿Cómo afecta a la aplicación? Sin espacio libre la instalación falla, y una app que guarda caché sin límite acaba siendo desinstalada. La documentación oficial avisa: «Internal storage has limited space for app-specific data. Use other types of storage if you need to save a substantial amount of data».

Ejemplo con dos dispositivos. El Moto G47 trae 128 GB, ampliables con microSD hasta 1 TB, y el iPhone 17 Pro Max llega a 2 TB sin poder ampliarse. Una app que guarda contenido sin conexión puede dejar sin espacio al primero, y en el segundo ni se nota.

4.4 Memoria RAM y potencia de procesamiento

¿En qué consiste? La RAM es el espacio donde se ejecutan los procesos y el procesador (SoC) es lo que los ejecuta. Android publica una tabla de umbrales de «mal comportamiento» (*Android vitals*) que depende de la RAM del dispositivo: en primer plano va de 2,00 GB (4 GB de RAM) a 4,25 GB (16 GB).

Valores o categorías. El criterio que uso es el de `isLowRamDevice()` (visto en el Apartado 1), porque decide cómo tengo que comportarme:

Gama	RAM física	`isLowRamDevice()`	Qué hacer
Baja	$\leq$ 4 GB	`true`	Cachés mínimos, sin `largeHeap`, imágenes reducidas, paginación, 2 hilos máx.
Media	6-8 GB	`false`	Comportamiento estándar
Alta	12 GB+	`false`	Margen para más trabajo concurrente

Cuadro 9. Clasificación por gama de RAM

¿Cómo afecta a la aplicación? Con poca RAM el sistema cierra la app antes y las animaciones o los filtros de imagen van a trompicones; con un procesador lento, además, se gasta más batería para hacer lo mismo.

Ejemplo con dos dispositivos. El Moto G47 tiene 8 GB de RAM reales (los 16 GB extra son virtuales) y un Dimensity 6300, y el Pixel 10 Pro XL tiene 16 GB y un Tensor G5. Según Android vitals, en primer plano el sistema tolera 2,25 GB a la app en el primero y 4,25 GB en el segundo.

4.5 Conectividad

¿En qué consiste? Es el conjunto de redes que puede usar el móvil, y es el recurso más variable de los cinco. Además hay un factor transversal: que la red sea **medida** (*metered*), es decir, que el usuario pague por datos.

Valores o categorías. Datos móviles (4G LTE y 5G), Wi-Fi (Wi-Fi 5, Wi-Fi 6 desde Android 9 y Wi-Fi 7, 802.11be, desde Android 13), Bluetooth (clásico y *Low Energy*, pensado para sensores y wearables con poco gasto de batería) y NFC (comunicación a menos de 4 cm: lector, emulación de tarjeta y entre dispositivos).

¿Cómo afecta a la aplicación? Android pide tres cosas a una app que use 5G: comprobar si la red es medida (`isActiveNetworkMetered()`), detectar si es 5G y estimar el ancho de banda. Una app de pagos con NFC no funciona sin chip NFC, y elegir Bluetooth clásico en vez de LE puede costar horas de autonomía.

Ejemplo con dos dispositivos. El Moto G47 (Wi-Fi 5, Bluetooth 5,4, sin UWB) y el Pixel 10 Pro XL (Wi-Fi 7, Bluetooth 6.0, UWB) no se comportan igual con la misma app: la transferencia de archivos es más lenta en el primero y una función de localización precisa por UWB solo puede existir en el segundo, así que necesita una alternativa.

4.6 Conclusión del apartado 4

Un desarrollador tiene que conocer estas configuraciones antes de diseñar porque la misma aplicación se comporta de forma distinta según la densidad, la versión de Android, el espacio, la RAM y la conectividad del móvil. Lo que funciona en un teléfono moderno puede cerrarse, verse borroso o no instalarse en uno antiguo. Por eso estos cinco datos conviene consultarlos en tiempo de ejecución (`DisplayMetrics`, `Build.VERSION.SDK_INT`, `StatFs`, `ActivityManager`, `ConnectivityManager`) en vez de suponerlos.

5 Establecimiento de perfiles de usuario

Hasta aquí he hablado de dispositivos; pero los usa gente con necesidades que no aparecen en ninguna ficha técnica. Cada perfil de la tabla describe a una persona *con* su móvil y su contexto, porque si se separan las dos cosas quedan dos listas de supuestos que no se hablan y luego el diseño falla. Los perfiles son ficticios.

5.1 Tabla de perfiles

Cuadro 10. Perfiles de usuario

	P1 Lucía	P2 Carmen	P3 Andrés	P4 Javier	P5 Samir
Edad	19 años	71 años	54 años	41 años	27 años
Ocupación	Estudiante universitaria	Jubilada	Administrativo	Electricista	Repartidor
Intereses y necesidades	Redes sociales, juegos y vídeo; rapidez y facilidad de uso	Llamadas, mensajería y citas médicas; interfaz clara	Banca y gestiones online; leer sin esfuerzo	Mensajería, mapas y gestiones desde el móvil; manejarlo con una mano	Navegación y mensajería con clientes; que la app no se cuelgue
Experiencia tecnológica	Alta	Baja	Media	Media	Media
Dispositivo	Gama alta reciente, Android 16, 5G	Gama media de hace 3 años, Android 11	Gama media, Android 14, lector de pantalla activado	Gama media, Android 15	Gama baja, Android 9, 3 GB de RAM, 32 GB, 4G intermitente, pantalla `hdpi`
Limitaciones de acceso	Ninguna	Le cuesta encontrar opciones y los gestos	Baja visión (retinopatía)	Temblor en las manos; pulsa mal botones pequeños	Poca memoria libre y datos limitados
Accesibilidad	Ninguna especial	Fuente grande y contraste alto	Texto ampliable, contraste, TalkBack	Botones grandes, sin gestos precisos	Ninguna, pero necesita ahorro de recursos
Cómo debe adaptarse la app	Respuesta fluida, modo oscuro, `targetSdk` al día	Pocas opciones por pantalla, un botón grande por acción, textos claros y botón de ayuda	Texto en `sp`, contraste $\geq$ 4,5:1, etiquetas para TalkBack, no depender solo del color	Objetivos de 48×48 dp con 8 dp de separación, sin gestos complejos, alternativas a arrastrar	Imágenes ligeras, sin `largeHeap`, paginación, caché acotada, *offline-first* y comprobar `isActiveNetworkMetered()`

Los perfiles P2, P3 y P5 son los que más se relacionan con los criterios del Apartado 4: un móvil de gama baja (4.4) suele traer poco almacenamiento (4.3), versión antigua (4.2) y pantalla `hdpi` (4.1). Por eso un perfil no es una categoría de personas, sino una combinación de persona, dispositivo y contexto.

5.2 Criterios de accesibilidad y diseño

Estos son los cinco parámetros concretos que más pesan al diseñar para los perfiles P2, P3 y P4:

- **Objetivo táctil de 48×48 dp como mínimo**, con 8 dp o más de separación. El área táctil no es lo que se ve: un icono de 24 dp puede llegar a 48 dp con *padding*.
- **Contraste de 4,5:1** en texto pequeño y de 3:1 en texto grande (18 pt, o 14 pt en negrita), según WCAG 2.2.
- **Texto que pueda crecer hasta el 200 %** sin romper el diseño (WCAG 1.4.4), para lo que hay

que escribirlo en `sp`.

- **Etiquetas en todo** (`contentDescription`) y un orden lógico, para que TalkBack pueda describir la pantalla.
- **Subtítulos y vibración** como alternativa al sonido, y compatibilidad con Bluetooth LE Audio (Android 13), pensado para audífonos.

5.3 Reflexión final

¿Por qué es importante conocer a los usuarios antes de desarrollar? Porque el dispositivo y la persona no se analizan por separado: un móvil antiguo con 3 GB de RAM y Android 9 (como el de Samir) es un caso muy distinto del móvil de gama alta con el que se prueba normalmente.

¿Qué características de diseño mejoran la accesibilidad? Objetivos táctiles de al menos 48 dp, contraste de 4,5:1, texto en `sp` que pueda crecer hasta el 200 %, etiquetas para TalkBack, no depender solo del color y ofrecer subtítulos o alternativas a los gestos.

¿Cómo adaptaría una aplicación para un dispositivo con recursos limitados? Imágenes ligeras y cargadas bajo demanda, caché acotada, `isLowRamDevice()` para activar un modo reducido, paginación en lugar de listas largas y diseño *offline-first* con descargas solo cuando la red no sea medida.

¿Qué problemas aparecen si se diseña solo para usuarios expertos? Interfaces con opciones escondidas, gestos que nadie descubre, textos pequeños y poca tolerancia a errores: Carmen, Andrés o Javier abandonan la app. La accesibilidad no es un extra: es la versión más exigente del diseño responsive, y si la app la cumple, funciona para todos los demás.

Conclusiones finales

Con esta actividad he aprendido que las limitaciones de un móvil (poca memoria, procesador limitado, batería finita y conexión intermitente) no impiden hacer una aplicación, pero obligan a tomar decisiones desde el principio. Entre las tecnologías, la diferencia que más me ha pesado es que el desarrollo nativo (Java, Kotlin, Swift) rinde más y llega a todo el hardware, mientras que el multiplataforma (Flutter, React Native) ahorra coste y tiempo a cambio de una capa intermedia; no existe una opción mejor para todos los casos. Con Android Studio la mayor dificultad fue la virtualización por hardware, que debe estar activada en la BIOS para que el emulador funcione, además del espacio en disco y de esperar a la sincronización de Gradle. Adaptar las aplicaciones a distintos dispositivos y usuarios es importante porque el mismo código se ejecuta en móviles de gama baja y alta, y lo usan personas con necesidades muy distintas. En mis propias aplicaciones consultaré en tiempo de ejecución lo que haga falta (`isLowRamDevice()`, `isActiveNetworkMetered()`), cuidaré la accesibilidad desde el diseño y probaré en un emulador de gama baja, no solo en uno potente.

Referencias bibliográficas

Todas las fuentes que aparecen en este trabajo son **documentación oficial del fabricante o de la plataforma** (Android Developers, Google Play Console, Apple, Material Design, W3C, Bluetooth SIG).

Las páginas citadas se han consultado el **`FECHA DE CONSULTA`**; las cifras de versiones, políticas de Google Play y umbrales de memoria cambian con frecuencia.

Fuentes de Android Developers (documentación técnica)

Android Developers. (2026). *Android vitals*. https://developer.android.com/topic/performance/vitals

Android Developers. (2026). *Support different screen densities*. https://developer.android.com/training/multiscreen/screendensities

Android Developers. (2026). *Screen sizes*. https://developer.android.com/training/multiscreen/screensizes

Android Developers. (2026). *Configuration.screenLayout*. https://developer.android.com/reference/android/content/res/Configuration

Android Developers. (2026). *Android API levels*. https://developer.android.com/tools/releases/platforms

Android Developers. (2026). *uses-sdk element*. https://developer.android.com/guide/topics/manifest/uses-sdk-element

Android Developers. (2026). *Behavior changes: all apps*. https://developer.android.com/about/versions/17/behavior-changes-all

Android Developers. (2026). *Storage overview*. https://developer.android.com/training/data-storage

Android Developers. (2026). *ActivityManager*. https://developer.android.com/reference/android/app/ActivityManager

Android Developers. (2026). *5G*. https://developer.android.com/develop/connectivity/5g

Android Developers. (2026). *Bluetooth*. https://developer.android.com/develop/connectivity/bluetooth

Android Developers. (2026). *Bluetooth LE Audio overview*. https://developer.android.com/develop/connectivity/bluetooth/ble-audio/overview

Android Developers. (2026). *Near Field Communication*. https://developer.android.com/develop/connectivity/nfc

Android Developers. (2026). *Wi-Fi*. https://developer.android.com/develop/connectivity/wifi

Fuentes de Android Studio (Apartado 3)

Android Developers. (2026). *Android Studio*. https://developer.android.com/studio

Android Developers. (2026). *Install Android Studio*. https://developer.android.com/studio/install

Android Developers. (2026). *Create a new Android Studio project*. https://developer.android.com/studio/projects/create-project

Android Developers. (2026). *Android Studio project templates*. https://developer.android.com/studio/projects/templates

Android Developers. (2026). *Run the Android Emulator*. https://developer.android.com/studio/run/emulator

Android Developers. (2026). *Configure hardware acceleration*. https://developer.android.com/studio/run/emulator-acceleration

Android Developers. (2026). *Create and manage Android virtual devices*. https://developer.android.com/studio/run/managing-avds

Android Developers. (2026). *Android Emulator troubleshooting*. https://developer.android.com/studio/run/emulator-troubleshoot

Android Developers. (2026). *Android UI development is Compose-first* [blog, 19/05/2026]. https://developer.android.com/blog/posts/android-ui-development-is-compose-first

Fuentes de accesibilidad y diseño

Android Developers. (2026). *Make your app more accessible*. https://developer.android.com/guide/topics/ui/accessibility/apps

Android Developers. (2026). *Principles for improving app accessibility*. https://developer.android.com/guide/topics/ui/accessibility/principles

Google. (2026). *TalkBack y accesibilidad en Android*. https://support.google.com/accessibility/android/answer/6006564

Google. (2026). *Use TalkBack*. https://support.google.com/accessibility/android/answer/6007100

Google. (2026). *Change font size and display size*. https://support.google.com/accessibility/android/answer/11183305

Google. (2026). *Target API level requirements*. https://support.google.com/googleplay/android-developer/answer/11926878

World Wide Web Consortium (W3C). (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. https://www.w3.org/TR/WCAG22/

Material Design 3. (2026). *Color contrast*. https://m3.material.io/foundations/designing/color-contrast

Fuentes de fabricantes

Samsung Electronics. (2026). *Galaxy S26 Ultra — Especificaciones técnicas*. https://www.samsung.com/es/smartphones/galaxy-s26-ultra/

Google. (2026). *Pixel 10 Pro XL — Especificaciones técnicas*. https://store.google.com/product/pixel_10_pro_specs

OnePlus. (2026). *OnePlus 13 — Especificaciones*. https://www.oneplus.com/es/13/specs

Apple Inc. (2026). *iPhone 17 Pro Max — Especificaciones técnicas*. https://www.apple.com/iphone-17-pro-max/technical-specs/

Motorola. (2026). *moto g47 — Especificaciones técnicas*. https://www.motorola.com/sa/en/p/phones/moto-g/47/pmipmjs44mz

Fuentes de lenguajes y frameworks del Apartado 2

Kotlin Foundation. (2026). *Kotlin documentation*. https://kotlinlang.org/docs/home.html

Oracle. (2026). *Java Documentation*. https://docs.oracle.com/en/java/

Flutter. (2026). *Flutter documentation*. https://docs.flutter.dev/

Meta Open Source. (2026). *React Native documentation*. https://reactnative.dev/docs/getting-started

Apple Inc. (2026). *Swift Documentation*. https://developer.apple.com/documentation/swift/

Mapa de fuentes por apartado

Apartado	Fuentes principales
1. Limitaciones	Android vitals · ActivityManager · Bluetooth (Android Developers) · fichas técnicas de los cinco fabricantes
2. Tecnologías	Documentación de Kotlin, Oracle, Flutter, React Native y Apple
3. Entorno	Android Studio · Install · Create project · Emulator · Hardware acceleration · AVDs · Troubleshooting del emulador
4. Config. dispositivos	Screen densities · Screen sizes · Configuration.screenLayout · API levels · uses-sdk · Target API (Google Play) · Storage · vitals · ActivityManager · 5G · Bluetooth · NFC · Wi-Fi · Behavior changes (Android 17)
5. Perfiles	Make your app more accessible · Principles for accessibility · TalkBack · Tamaño de fuente · WCAG 2.2 · Material Design 3 (contraste) · Bluetooth LE Audio

Cuadro 11. Mapa de fuentes por apartado